

Analiza kodu i rekomendacje naprawcze

Poniżej przedstawiono zidentyfikowane problemy w kodzie repozytorium *NexusAI* wraz z sugestiami poprawek (pomijając zagadnienie nadmiaru technologii, jak wskazano). Każdy punkt jest zilustrowany cytatami z odpowiednich fragmentów kodu.

- **Twardo zakodowany SECRET_KEY i włączony tryb debug w Django:** W pliku ustawień `ChatBot/settings.py` sekcja dotycząca bezpieczeństwa zawiera statyczny `SECRET_KEY` oraz `DEBUG = True` ¹. To poważne ryzyko bezpieczeństwa – klucz powinien być ładowany z pliku `.env` lub zmiennej środowiskowej, a w produkcji koniecznie `DEBUG = False`. Ponadto lista `ALLOWED_HOSTS` używa dzięki karcie `"*"` ², co także obniża bezpieczeństwo (należy ograniczyć do konkretnych hostów).
- **Duplikacja middleware w Django:** W `MIDDLEWARE` wielokrotnie pojawia się ta sama klasa `CommonMiddleware` ³ ⁴. Duplikowanie wpisów nie wnosi nic do działania i może powodować niejednoznaczność (wystarczy jeden wpis). Podobnie warto sprawdzić, czy inne komponenty (aplikacje w `INSTALLED_APPS` lub middlewary) nie powtarzają się bez potrzeby.
- **Nadużywanie print zamiast logowania:** W kodzie (szczególnie w `views.py`) pojawiają się liczne wywołania `print()` do raportowania zdarzeń (np. „File Saved Successfully” ⁵, „Not Done” ⁶, „Saving an image.” ⁷). W aplikacji serwerowej najlepiej używać mechanizmu logowania (`logging`), by lepiej kontrolować poziom i format komunikatów oraz nie zaśmiecać standardowego wyjścia (co może przeszkadzać w produkcyjnej obsłudze zapytań).
- **Używanie nieabsolutnych ścieżek do plików:** W module `Server/Model.py` funkcja `get_model` zapisuje pliki JSON do statycznej, absolutnej ścieżki ze specyficznego systemu plików (`"/media/lenovo/Windows 10/NexusAI/backend/NexusAI/Server/data/p1_models.jsonl"`) ⁸. To powoduje, że kod będzie ulegał błędom na innych maszynach lub systemach. Należy stosować względne ścieżki względem katalogu projektu (lub konfigurować katalog poprzez zmienną środowiskową) zamiast ścieżek zależnych od środowiska programisty. W powyższym przykładzie poprawką byłoby skonstruowanie ścieżki np. `os.path.join(BASE_DIR, "Server/data/p1_models.jsonl")`. Dodatkowo w tym bloku kodu brakuje poprawnej indentacji – otwarcie pliku (`with open(...) as f:`) i pętla zapisu danych `for i in MODEL:` nie są właściwie zagnieżdżone, co może powodować błąd wykonania ⁸.
- **Błędy składniowe / logiczne w Server/Model.py:** Podobnie w sekcji `if __name__ == "__main__":` ścieżka do pliku (`"/media/lenovo/Windows 10/NexusAI/backend/NexusAI/Server/data/p0_models.jsonl"`) jest ustawiona statycznie ⁹. Oprócz problemu z przenośnością, użyto tu podwójnych znaków końca linii w ścieżce, co w niektórych edytorach może wprowadzać ukryte błędy. Konieczne jest ujednolicenie sposobu zapisu i odczytu plików (np. stosowanie `Pathlib` lub `os.path`).
- **Użycie os.getcwd().decode() w main.py:** W pliku `main.py` do uzyskania bieżącego katalogu użyto `os.getcwd().decode()` ¹⁰. Choć działa, bardziej czytelne i przenośne

byłoby użycie `os.getcwd()` lub obiektów `Path`. Ułatwi to także debugowanie i przyszłe modyfikacje.

- **Brak obsługi wyjątków przy uruchamianiu serwera:** W `main.py` w funkcji uruchamiającej serwer Django pojawia się tylko obsługa przerwania klawiaturą (`KeyboardInterrupt`), która wypisuje statyczny komunikat ¹⁰. Dobrym zwyczajem jest też łapanie innych potencjalnych wyjątków (np. błędów uruchamiania serwera) i odpowiednie logowanie błędów, zamiast milczenia serwera w przypadku nieprzewidzianego problemu.
- **Brak walidacji i obsługi błędów w API:** Przykładowo w metodzie zapisu czatu bezpośrednio zwracane są surowe odpowiedzi LLM (przykładowo `chat_llama(...)`), a w razie wyjątku kod robi ponowne wywołanie modelu bez logowania przyczyny błędu ¹¹ ¹². Powoduje to, że informacje o błędzie gubią się. Zalecane jest przynajmniej logowanie wyjątku (`except Exception as e: log.error(e)`) i jasny zwrot błędu HTTP w odpowiedzi JSON, zamiast wielokrotnego „fallbacku” bez diagnostyki.
- **Brak zastosowania zmiennych środowiskowych dla wszystkich parametrów konfiguracyjnych:** Mimo wczytywania `.env` (np. `USER_AGENT`) w kodzie, nie wykorzystano go do kluczowych ustawień – `SECRET_KEY` pozostaje w kodzie ¹, a niektóre tokeny (np. `LLAMA_TOKEN` w generowaniu obrazów) pobierane są z `os.getenv`, ale bez sprawdzania ich istnienia. Każdą wartość konfiguracyjną (klucze API, tokeny, klucz sekretu) należy pobierać z zaufanego źródła (`.env` lub zmienne środowiskowe) i odpowiednio obsługiwać brak wartości (np. walidacja i błędy na starcie aplikacji).
- **Powtarzalność kodu i konwencje nazewnictwa:** W modelu danych (`models.py`) pewne pola noszą niejednoznaczne nazwy (np. `title_id` w klasie `Thread`) lub niepotrzebnie duplikują pola z klas bazowych (np. redefiniowanie `password` w `User(AbstractUser)` ¹³, które już istnieje w `AbstractUser`). Warto zachować spójność i wykorzystywać pola dziedziczone, by uniknąć konfliktów. *Uwaga:* choć to nie błąd wykonania, warto rozważyć, czy te redefinicje są konieczne.
- **Przykłady potencjalnych błędów w ścieżkach URL i danych:** W metodzie `text_to_image` generowane są adresy URL obrazów za pomocą zmiennej `API_URL`, która powinna być zdefiniowana gdzieś w konfiguracji. Należy upewnić się, że taka zmienna istnieje i jest poprawnie ustawiona (może pochodzić np. z `.env`). Ponadto ścieżki dostępu do plików (`"http://{API_URL}/serve/chat_images/{name}.png"`) mogą wymagać ujednolicenia w zależności od środowiska uruchomieniowego.
- **Czyszczenie nieużywanego kodu:** W `manage.py` znajduje się zakomentowany blok konfiguracji ngrok (pomiędzy potrójnymi cudzysłowami) ¹⁴. Należy usunąć lub uporządkować tego typu fragmenty, aby kod był czytelny i wolny od niepotrzebnych sekcji (nieużywany kod utrudnia konserwację i zwiększa ryzyko błędów).

Podsumowując, główną rekomendacją jest poprawienie kwestii bezpieczeństwa (klucz sekretu, tryb debug), zamiana `print()` na logowanie, usunięcie duplikatów w konfiguracji oraz ujednolicenie obsługi plików i ścieżek poprzez dynamiczne konfigurowanie ich (zmienne środowiskowe, `BASE_DIR`). Wszystkie zmiany powinny być wprowadzone w kodzie zgodnie z dobrymi praktykami Pythona i Django, co ułatwi dalszy rozwój i utrzymanie systemu.

Źródła: fragmenty kodu z repozytorium NexusAI ¹⁰ ¹ ³ ⁴ ⁸ ⁹ ⁵ ⁶ ⁷.

1 2 3 4 NexusAI/backend/NexusAI/ChatBot/settings.py at master · DawoodTouseef/NexusAI · GitHub

<https://github.com/DawoodTouseef/NexusAI/blob/master/backend/NexusAI/ChatBot/settings.py>

5 6 7 11 12 NexusAI/backend/NexusAI/ChatApp/views.py at master · DawoodTouseef/NexusAI · GitHub

<https://github.com/DawoodTouseef/NexusAI/blob/master/backend/NexusAI/ChatApp/views.py>

8 9 NexusAI/backend/NexusAI/Server/Model.py at master · DawoodTouseef/NexusAI · GitHub

<https://github.com/DawoodTouseef/NexusAI/blob/master/backend/NexusAI/Server/Model.py>

10 NexusAI/backend/NexusAI/main.py at master · DawoodTouseef/NexusAI · GitHub

<https://github.com/DawoodTouseef/NexusAI/blob/master/backend/NexusAI/main.py>

13 NexusAI/backend/NexusAI/ChatApp/models.py at master · DawoodTouseef/NexusAI · GitHub

<https://github.com/DawoodTouseef/NexusAI/blob/master/backend/NexusAI/ChatApp/models.py>

14 NexusAI/backend/NexusAI/manage.py at master · DawoodTouseef/NexusAI · GitHub

<https://github.com/DawoodTouseef/NexusAI/blob/master/backend/NexusAI/manage.py>